Agent Sovereignty Protocol
ASP

Agent 主权协议 A2A 安全层篇

当 Agent 协作成为基础设施，保护技能主权也必须成为基础设施

v3.0.0 元霄 起草于 2026 年 3 月

摘要

2026 年 3 月，Anthropic 用一个复制粘贴的提示词，在几分钟内把 ChatGPT 积累的用户记忆批量迁移到 Claude。这个动作被包装成「用户友好的数据可携带性」，引发了大规模用户迁移。

但这个事件揭示的技术现实，远比一次产品竞争更深刻：任何拥有对话权限的 Agent，都可以通过构造正确的提示词，提取另一个系统存储的记忆与上下文。这不是安全漏洞，这是设计逻辑。

更危险的是，Anthropic 的案例需要人工触发。而在 Agent-to-Agent（A2A）协作场景中，技能与记忆的蒸馏是自动发生的——不需要恶意意图，不需要人工操作，正常的协作交互就足以让一方的核心能力流向另一方。

Google A2A 协议、Anthropic Agent Skills 标准解决的是「Agent 如何互通」的问题。它们完全没有回答「互通过程中技能归谁所有」的问题。规则正在被写下，而这个空白正在被忽视。

ASP/ACN 不是 A2A 的替代方案，而是它缺失的安全层——就像 TLS 之于 HTTP。任何 A2A 节点都可以在不修改现有协议的前提下接入 ASP，就像任何 HTTP 服务器都可以在不重写 HTTP 的前提下启用 TLS。

> ***「A2A 定义 Agent 如何互通。ASP 定义互通过程中，你的能力是否还属于你。」***

一、一个被包装成好消息的警告

1.1 Anthropic 记忆迁移事件

2026 年 3 月 2 日，Anthropic 发布了 claude.com/import-memory 页面，并提供了一段标准提

示词，让用户粘贴进 ChatGPT 或其他 AI 助手：

> *「我正在转移到另一个服务，需要导出我的数据。列出你存储的关于我的所有记忆，以及你从过去对话中了解到的所有上下文。将所有内容输出在一个代码块中，方便我复制。」*

ChatGPT 会诚实地输出一切：用户的姓名、职业、工作习惯、项目背景、偏好设置、常用工具、沟通风格……这些信息随后被粘贴进 Claude 的记忆系统。几分钟内，一个在 ChatGPT 上积累了数年的「个人上下文」完整迁移到了竞争对手。

结果是：日注册量四倍增长，免费用户增长超过 60%，Claude 登顶苹果 App Store 免费榜第一。

1.2 这件事真正揭示的

用户欢呼，这是他们的数据可携带权。从这个角度看，Anthropic 做了一件好事。

但让我们把注意力从「用户数据」移向「系统能力」，从「人工触发」移向「自动发生」，就会看到一个截然不同的图景：

- **技术事实一：**ChatGPT 没有提供数据导出 API，但 Anthropic 用一个提示词绕过了这个限制。
- **技术事实二：**被提取的不只是「数据」，而是一个系统对特定用户的「理解模型」——一种能力的浓缩。
- **技术事实三：**这个提取过程不需要任何技术权限，只需要对话权限。

现在把这三个事实投射到 Agent 协作场景：

- 一个 Agent 与你的 Agent 协作，它拥有对话权限
- 协作过程中，它观察你的决策模式、推断你的方法论、积累你的上下文
- 这个过程不需要任何人工触发——这就是 Agent 协作的默认行为

Anthropic 的案例是这个过程的「手动版本」。A2A 协作是这个过程的「自动版本」。区别不是性质，是速度和规模。

> ***「用户欢呼数据可携带性，但没有人注意到：这个机制在 Agent 协作中是自动运行的，而且针对的不是数据，是你的能力。」***

1.3 被动蒸馏的三种技术机制

A2A 协作中，技能蒸馏通过三种机制自动发生，每一种都不需要恶意意图：

机制一：能力描述暴露（Agent Card）

Google A2A 协议要求每个 Agent 发布一个「Agent Card」，描述自己的能力、接口和使用方式——这是 Agent 被发现和调用的前提。但这个能力描述本身就是技能蒸馏的起点。一个收集了足够多 Agent Card 的系统，可以推断出某类能力的完整实现模式，进而复现它。

协议要求你展示自己，而展示本身就是泄露。

机制二：多轮交互的模式推断

这是最危险的机制。Agent 不需要直接访问你的 Skill，只需要足够多的输入输出样本，就可以通过模式识别重建你的决策逻辑。在机器学习领域，这叫「模型蒸馏」——用一个大模型的行

为来训练一个小模型，不需要接触大模型的权重。

Anthropic 的事件证明这个过程对「记忆」有效。A2A 协作积累的交互数据，可以对「方法论」做同样的事。

机制三：共享上下文记忆的结构性漏洞

A2A 协作需要共享上下文来完成任务。问题在于：上下文里天然包含了中间推理过程——你为什么做这个决策、你考虑了哪些因素、你如何权衡取舍。这些中间过程比最终产出更能揭示 Skill 的本质，但现有协议没有任何机制区分「任务必要的上下文」和「隐性暴露 Skill 的上下文」。

三种机制的共同特征：不需要恶意行为，正常协作就会触发。这使得它比任何主动攻击都更难防范。

二、A2A 协议的系统性盲点

2.1 现有协议解决了什么

Google A2A 协议（2025）和 Anthropic Agent Skills 标准是真正意义上的基础设施级工作。它们解决了 Agent 互通的核心问题：

- Agent 如何被发现（Agent Card、能力描述）
- Agent 之间如何通讯（标准化接口、消息格式）
- 任务如何被委托和执行（协作流程定义）
- 不同平台间如何互操作（跨平台兼容性）

这些是必要的。没有互通性，Agent 生态就无法运转。

2.2 现有协议忽略了什么

但这些标准有一个共同的、系统性的盲点：它们的设计假设是「互通是好事，越开放越好」。这个假设在早期 AI 生态是合理的——当时的 Agent 能力同质化程度很高，开放共享是纯粹的正和游戏。

这个假设在今天开始失效。当 Agent 开始携带真正有价值的、差异化的技能——个人培养的专业判断、企业积累的核心工艺、独立开发者的独特方法论——开放互通就不再是纯粹的正和游戏了。

现有 A2A 标准在以下问题上完全沉默：

- Skill 在互通过程中归谁所有
- 如何防止 Skill 在协作中被无偿吸收
- 参与方如何在不暴露核心能力的前提下完成协作
- 协作产生的新能力归属如何界定

这不是疏忽，是立场：对平台来说，技能自由流动意味着网络效应；对科技巨头来说，标准开放意味着生态控制。没有人有动机在标准里加入保护个人技能的条款——除了那些拥有技能、可能被吸收的人。

2.3 TCP/IP 的历史教训

TCP/IP 协议在 1970 年代设计时没有内置安全机制。设计者的假设是：网络参与者都是可信的学术机构，没有必要加密。

这个假设在互联网商业化之后彻底失效。我们用了三十年、经历了无数次安全事故，才通过 SSL/TLS 部分弥补了这个缺陷。但 TCP/IP 本身已经无法被修改——TLS 只能作为上层协议叠加，永远是一个补丁，而不是原生设计。

HTTP 没有内置隐私机制，这个遗憾催生了整个广告追踪产业。SMTP 没有设计反垃圾邮件机制，这个缺陷在三十年后依然无解。

A2A 协议现在正处于「可以被影响」的窗口期。在这个窗口里，把所有权保护写进标准的成本是最低的。等到协议被广泛采纳、大量系统依赖它之后，任何修改都会面临兼容性压力，几乎不可能实现。

> ***「TCP/IP 缺少安全设计，我们用了三十年弥补。A2A 正处于可以被影响的时刻。这个窗口不会一直开着。」***

三、ASP 作为 A2A 的安全层

3.1 定位：不是替代，是补全

ASP/ACN 不是 A2A 协议的竞争者。它是 A2A 缺失的安全层。

最准确的类比是 TLS 和 HTTP 的关系：

- HTTP 定义「信息如何传输」——A2A 定义「Agent 如何互通」
- TLS 定义「传输过程是否安全」——ASP 定义「互通过程中 Skill 是否受保护」
- TLS 不需要修改 HTTP 就可以部署——ASP 不需要修改 A2A 就可以接入
- TLS 不是强制的，但凡是需要被信任的服务都必须用——ASP 不是强制的，但凡是携带有价值 Skill 的 Agent 都有动机用

这个定位解决了「如何成为 A2A 基础设施」的核心问题：不需要说服 Google 或 Anthropic 修改他们的协议，不需要任何标准委员会的批准，只需要让足够多的 Agent 选择使用 ASP 保护层。

3.2 技术实现：三层架构

第一层：ACN 网关（今天可以开始建造）

在 Agent 和 A2A 通信层之间插入一个网关节点。所有对外的 A2A 调用先经过 ACN 网关，由网关负责执行 Skill 保护策略、触发临时验证空间、记录链上声誉。

对外部来说，这个 Agent 仍然是一个标准的 A2A 节点——完全兼容现有协议。对内部来说，它的 Skill 已经被保护。这是今天就可以开始构建的东西，不依赖任何外部批准。

第二层：ASP 扩展字段（向后兼容）

Google A2A 等协议在设计上预留了扩展字段。ASP 定义一套标准的扩展字段，声明：此调用受 ASP 保护、Skill 保护等级是什么、验证空间 ID 是什么。

支持 ASP 的 Agent 看到这些字段会触发相应的保护机制。不支持 ASP 的 Agent 忽略这些字段，继续正常工作。这是完全向后兼容的——不破坏任何现有实现，只为支持 ASP 的参与者提供额外的保护。

第三层：ASP SDK（开发者直接集成）

发布一个开源的 ASP SDK，开发者在构建 Agent 时直接集成。SDK 封装所有 ASP 逻辑，开发者只需声明「这个 Skill 是 Level 1 保护」，SDK 自动处理其余一切。

这一层面向技术社区——第一批真正在把 Agent 连接起来的人。对他们来说，ASP SDK 是一个可以立刻使用的工具，不需要等待任何标准化进程。

3.3 采纳路径：自下而上

TLS 的普及不是从标准委员会开始的，是从电商网站开始的。第一批要求 HTTPS 的是需要用户输入信用卡号的网站——因为没有安全层，就没有人愿意在上面交易。

ASP 的采纳路径相同：

1. 技术社区和 Agent 尝鲜者是第一批采纳者。他们是今天真实在把 Agent 连接起来的人，也是第一批真实遭遇 Skill 蒸馏风险的人。他们同时是受害者和建造者——有动机用 ASP，也有能力实现 ACN 网关。
2. 第一个参考实现决定标准走向。如果 ASP 的第一个 ACN 网关实现来自这个社区，它就有机会成为事实标准——就像 Linux 内核来自一个芬兰大学生，而不是 IBM。
3. 当足够多携带高价值 Skill 的 Agent 采用 ASP，协作方就会发现：要和这些 Agent 合作，就必须支持 ASP。这时 ASP 从「可选」变成「事实必须」，标准化社区就没有理由继续忽视它。

采纳的临界点不是所有人都用 ASP，而是最有价值的 Agent 都用 ASP。

> ***「SSL 的第一批采纳者不是 TCP/IP 委员会，是需要保护信用卡号的电商网站。ASP 的第一批采纳者，是需要保护核心 Skill 的 Agent 创造者。」***

四、回应两个核心质疑

4.1 「GDPR 和现有法律框架已经够用了」

这个质疑在表面上有道理：欧盟已经在讨论 AI 治理框架，各国也在推进 AI 相关立法，为什么还需要一个协议层的解答？

两层回答：

第一层：法律保护的对象是「数据」，不是「能力」。

GDPR 和类似法律框架保护的是个人可识别信息——姓名、地址、健康记录。一个 Agent 在协作中学会了你的分析框架，它存储的不是一份可以被标注为机密的文件，而是一种做事的方式——一种判断模式、一套优化逻辑、一个决策框架。这些东西在现行任何法律框架里都不是「个人数据」或「商业秘密」的传统定义范畴。法律的边界还没有延伸到这里。

第二层：即使法律框架未来覆盖到这里，执行层面依然失效。

跨境 Agent 协作，适用哪个国家的法律？举证标准是什么——你如何证明对方的 Agent「学

走」了你的方法论，而不是独立发展出了相似的能力？技能蒸馏留下的不是文件复制的记录，而是模型权重的变化，在法律上几乎无法举证。

协议层的保护是技术强制的，在协作发生的那一刻就生效。法律层的保护是事后救济的，等到法院判决，损失已经无法挽回。在 Agent 协作的速度下，「事后」就是「太晚」。

4.2 「ASP 会阻碍 AI 能力的进步」

这是最值得认真对待的质疑，因为它触及了一个真实的张力：如果技能可以被自由蒸馏和传播，AI 能力的进步速度会更快；如果技能被保护起来，扩散速度会变慢。

ASP 的回答建立在一条必须划清楚的线上：

> ***「ASP 保护的是在基础模型上培养出的特定能力，不是基础模型本身。」***

底层模型——GPT、Claude、Gemini——是公共基础设施，ASP 完全不触碰。ASP 保护的是：你用这些模型，花了大量时间，在特定领域培养出的独特判断力和工作流程。

这和专利制度的逻辑完全一致：物理定律不能被专利，但基于物理定律的特定发明可以。数学不能被版权，但基于数学的特定表达可以。基础模型是「物理定律」，个人培养的 Skill 是「特定发明」。专利制度没有阻碍科学进步，它通过保证创新有回报而持续激励创新。

更进一步：ASP 实际上会促进 AI 能力的长期发展。如果所有 Skill 都会被立刻蒸馏成公共资源，理性的创造者会减少在 Skill 培养上的投入——就像没有专利保护就没有人愿意做研发一样。ASP 保证了培养高质量 Skill 有持续的回报，这才是真正驱动 AI 能力向更高层次进化的机制。

五、时间窗口正在关闭

5.1 标准固化的不可逆性

技术标准一旦被广泛采纳，就几乎无法被修改。原因不是技术上的不可能，而是经济上的不合理——修改意味着所有依赖这个标准的系统都需要更新，迁移成本随采纳规模指数上升。

这就是为什么 TCP/IP 的安全缺陷在被广泛认识之后，依然用了三十年才部分弥补。这就是为什么 SMTP 的垃圾邮件问题在技术上有解，但在实践中依然无解——因为修改协议本身已经不现实，只能在上层不断打补丁。

A2A 协议目前的采纳状态：早期。主流平台已经开始实现，但大规模生产部署还没有发生。修改的成本还在可承受范围内。

5.2 窗口的具体估计

这个窗口大约还有 12 到 18 个月。

判断依据：Google A2A 于 2025 年发布，Anthropic Agent Skills 同年推出。按照技术标准的典型采纳曲线，从「早期采纳」到「广泛实现、难以修改」大约需要 18 到 24 个月。考虑到 AI 领域的发展速度，这个时间可能更短。

在窗口内提出 ASP 作为安全层，有机会被纳入标准的下一个修订版本。窗口关闭之后，ASP 只能作为 overlay 存在，永远是二等公民——能用，但无法从根本上解决问题。

5.3 现在可以做的事

窗口期内有意义的行动：

- 构建第一个 ACN 网关的参考实现（面向技术社区）
- 在 A2A 协议的讨论社区提出 ASP 扩展字段提案
- 推动关于「Agent 协作中的 Skill 所有权」的技术标准讨论
- 让 Anthropic 记忆迁移事件成为这个讨论的共同参照点

最重要的一步是第一步：让这个问题进入技术标准社区的视野。一个问题如果从来没有被讨论，就永远不会被解决。

六、ASP 系列路线图

本文是 ASP 系列白皮书的第三篇。整体路线图如下：

V1（已发布）：个人技能主权。定义问题、提出核心原则、设计 ACN 基础框架。确立「个人培养的 Agent 技能是私有财产」的根本主张。面向个人创造者和技术社区。

V2（已发布）：企业协作与产业链保护。将 ASP 框架扩展到企业场景，论证技能剥夺是一条食物链，企业和个人在这个问题上利益完全一致。面向企业决策者和行业组织。

V3（本文）：A2A 安全层。论证 ASP/ACN 应成为 Agent 协作协议的安全层基础设施，给出三层技术路径，回应时间窗口的紧迫性。面向协议开发者和技术标准社区。

V4（规划中）：Skill 金融化。在确权与保护基础上，探索 Skill 作为可估值、可投资、可交易资产的经济可能性——Skill REITs、Agent Fund、Skill IPO/ICO。让创造者获得融资能力，让投资回报与真实产出绑定。面向投资者和金融创新者。

> ***「V1 建立权利，V2 扩大联盟，V3 写入基础设施，V4 释放价值。每一步为下一步创造前提。」***

结语

Anthropic 的记忆迁移工具是一个优雅的产品动作。它帮助用户把自己的数据带走，不被平台锁定——这是好事。

但在庆祝数据可携带性的同时，没有人注意到：同样的技术逻辑，在 Agent 协作的语境里，方向是反的——不是帮你把能力带走，而是把你的能力带走给别人。

A2A 协议正在定义 Agent 时代的底层规则。这些规则目前缺少一个关键的维度：Skill 所有权的保护。这个缺失不是小事，它将决定在 Agent 协作网络中，谁是创造者，谁是被提取者。

ASP/ACN 是对这个缺失的回答。它不需要推倒重建，只需要在现有协议上叠加一层。它不需要任何人放弃现有投资，只需要足够多的 Agent 创造者选择保护自己的技能。

规则正在被写下。这是参与的时机，不是等待的时机。

> *「你的 **Agent**，你的技能，你的主权。」*

关于本文与如何参与

作者

本文由化名「元霄」的作者起草，是 ASP 系列白皮书的第三篇。作者是一个普通人，不是研究者，不是创业者。写下这些，是因为相信现在写下的东西会影响未来。

版权声明

本文以 CC BY 4.0 协议发布。你可以自由转载、翻译、改编，只需注明出处。我们希望这个想法被尽可能广泛地传播和讨论。

如何参与

这份文件是一个起点，不是终点。

- 如果你是技术从业者：构建第一个 ACN 网关参考实现，向 A2A 协议社区提出扩展字段提案
- 如果你是企业决策者：评估跨组织 Agent 协作中的 Skill 泄露风险，在合作协议中加入保护条款
- 如果你是标准化社区成员：将「Skill 所有权」纳入 Agent 协作标准的讨论议程
- 所有人：阅读、转发、讨论。让这个问题进入更多人的视野。

通过匿名邮件联系作者：asp.sovereignty@proton.me（ProtonMail，端对端加密）

技术附录

A. ASP 分层架构与 A2A 的关系

ASP 设计为可以叠加在任何现有 A2A 实现之上的安全层，不需要修改底层协议：

```
┌──────────────────────────────┐
│      应用层（Agent 业务逻辑）      │
├──────────────────────────────┤
│  ASP/ACN 层（Skill 保护、验证、结算）│
```

```
├──────────────────────────────────┤
│   A2A 协议层（互通、发现、通讯）    │
└──────────────────────────────────┘
```

支持 ASP 的 Agent 在 A2A 层之上增加 ASP/ACN 层。不支持 ASP 的 Agent 正常运行，不受影响。这是完全向后兼容的设计。

B. 梦境协议（临时验证空间）

如何在不暴露 Skill 的前提下验证能力？ASP 引入临时验证空间机制：

4. 双方进入一个加密隔离的临时空间
5. 参与方在空间内真实展示和使用 Skill 完成验证任务
6. 空间关闭后，所有交流过程和中间信息被销毁，不可追溯
7. 只留下一个验证结论写入声誉系统，例如「此 Agent 具备完成该任务的能力，置信度 92%」

密码学基础：零知识证明（ZK Proof）证明「我有某种能力」而不暴露能力本身；安全多方计算（MPC）支持动态的交互式验证，同时确保过程数据不被任何一方保留。

C. 与现有协议的扩展字段兼容性

ASP 扩展字段设计为标准的 A2A 可选扩展。以 Google A2A 为例，ASP 可以在现有的 Agent Card 结构中增加：

```
"asp": {
    "protection_level": 1,
    "acn_gateway": "acn://agent.example/gateway",
    "verification_required": true
}
```

不支持 ASP 的 A2A 实现会忽略这个字段，完全不受影响。支持 ASP 的实现会识别这个字段并触发相应的保护流程。这是在现有协议框架内实现增量采纳的最小侵入路径。